

Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

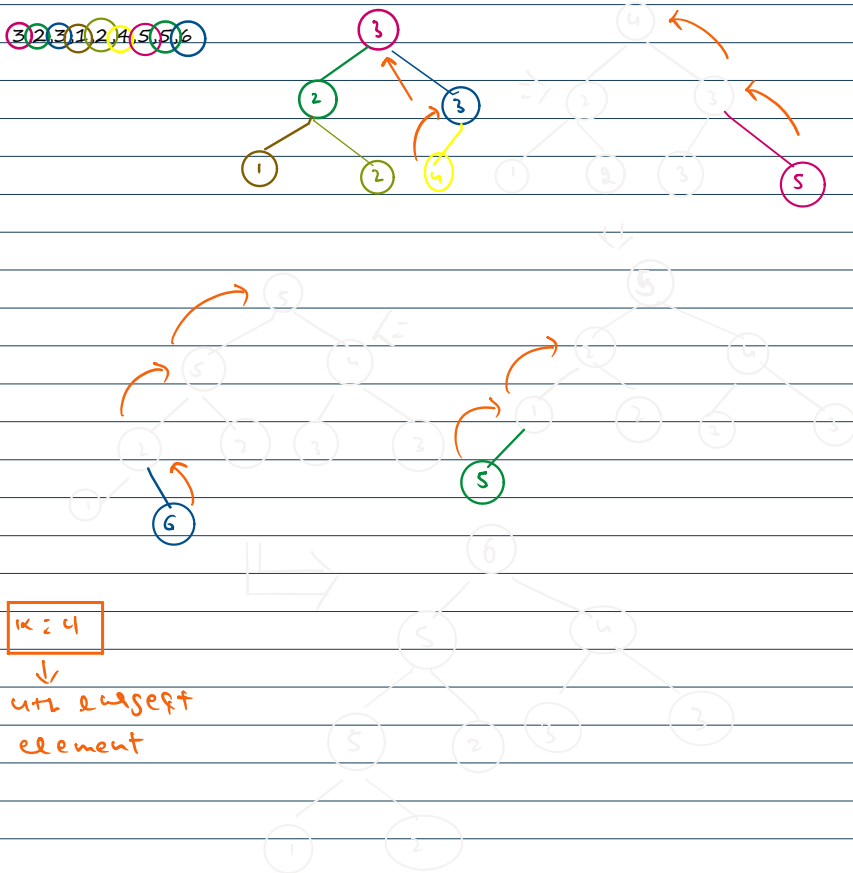
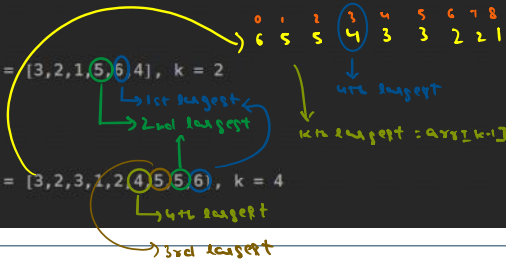
Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,6]`, `k = 4`

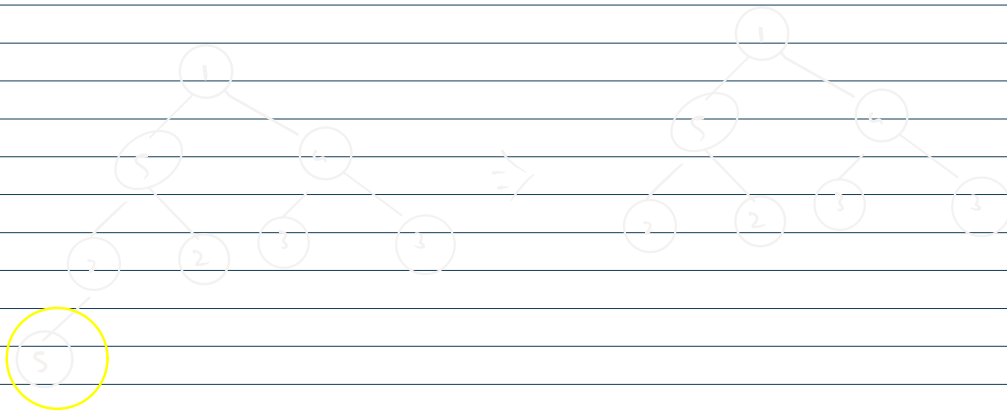
Output: 4



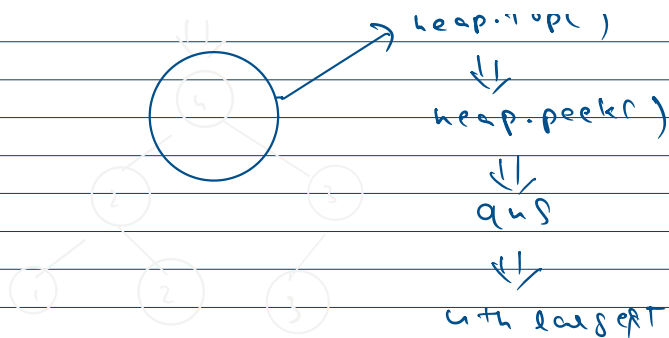
$k = 4$

↓
4th largest
element





Heap.top()
 ↓↓
 ...



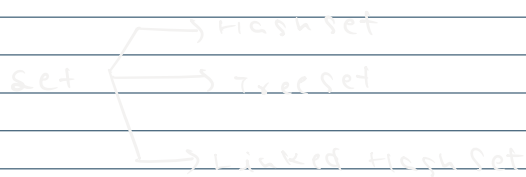
Set

HashSet

stores unique elements only.

no guarantee of ordering

o(1) $\Rightarrow$ avg. time insertion, searching, remove



HashSet

1. Implementation $\Rightarrow$ hash table
2. o(1) $\Rightarrow$ insertion, deletion, searching
3. no order of elements.
4. accepts one null value.
5. lowest memory usage
6. remove duplicates hashcode + equals.

Tree Set

1. Implementation $\Rightarrow$ self balancing BST $\Rightarrow$ (Red Black Tree)
2. o(log n) time operations.
3. Sorted ascending order of elements
4. null value is not allowed.
5. custom comparator can be used.

Linked HashSet

1. Implementation $\Rightarrow$ hash table + doubly linked list
2. Insertion order preserved
3. slightly slower than HashSet but faster than TreeSet.
4. allows one null value.

Hash Table

10	
25	
5	4 → 10
35	
300	
35	
37	
17	
14	
89	
30	

300

10 (key)
add

hashfunction

hashkey

35

4 (key)
add

hashfunction

hashkey

35

collision

300 (key)
add

hashfunction

hashkey

87

300 ← key

search

hashfunction ⇒ hashkey = 87

Frequency of elements

arr → [1, 5, 10, 5, 1, 10, 5, 2, 3, 5, 0]

✓ 1 : 1 2
✓ 5 : 1 2 3 4
✓ 10 : 1 2 3
2 : 1
3 : 1

{
1 : 2
5 : 4
10 : 3
2 : 1
3 : 1
}

map
Integer → Integer
key → value
element → count

arr1 → {5, 5, 5, 2, 2, 3, 0, 8}
arr2 → {5, 5, 5, 2, 2, 1, 2, 8, 7}

map2 → {
5 : 1 1 0
2 : 1 2 1
1 : 1 1
0 : 1 0
7 : 1
}

5, 5, 2, 2, 8

→ output

array

ArrayList

array

arr1 → {5, 5, 5, 2, 2, 3, 0, 8}